

Klasa String Buffer

Zbigniew Marszalek

Klasa *StringBuffer*

Klasa *StringBuffer* jest rozszerzeniem klasy *String*. Łańcuchy w klasie *String* dziedziczyły po języku C założenie o niezmienniej liczbie przechowywanych znaków. Dlatego zaprojektowano nową klasę *StringBuffer* umożliwiającą zmianę zawartości obiektu łańcucha. Nowa klasa zawiera wiele użytecznych metod do manipulowania ciągiem znaków zapisanym w obiekcie, niedostępnych w klasie *String*.

Konstruktory klasy *StringBuffer*

Klasa *StringBuffer* definiuje cztery konstruktory.

`StringBuffer()`

`StringBuffer( int size )`

`StringBuffer( String str )`

`StringBuffer(CharSequence chars )`

Domyślny konstruktor rezerwuje miejsce dla 16 znaków. Klasa *StringBuffer* zawsze rezerwuje miejsce dla większej liczby znaków, ponieważ alokacja jest kosztowna. Co więcej, częste alokacje powodują fragmentację pamięci. Rezerwacja miejsca dla kilku dodatkowych znaków redukuje potrzebę częstej alokacji.

Metoda *length()* i *capacity()*

Metoda *length()* zwraca aktualny rozmiar tekstu w obiekcie `StringBuffer`. Metoda *capacity()* zwraca aktualny obszar zaalokowany przez obiekt. Składnia metod:

int `length()`

int `capacity()`

```
StringBuffer sb = new StringBuffer("Witaj Świecie");  
System.out.println("Bufor = " + sb);  
System.out.println("Długość = " + sb.length());  
System.out.println("Pojemność = " + sb.capacity());
```

run:

Bufor = Witaj Świecie

Długość = 13

Pojemność = 29

Metoda *ensureCapacity()* i *setLength()*

Do zalokowania miejsca na znaki dla już utworzonego obiektu *StringBuffer* służy metoda *ensureCapacity()*. Składnia:

```
void ensureCapacity( int capacity)
```

Do ustawienia długości bufora obiektu *StringBuffer* służy metoda *setLength()*. Składnia:

```
void setLength( int len)
```

Metoda *charAt()* i *setCharAt()*

Do pobrania pojedynczego znaku z obiektu *StringBuffer* służy metoda *charAt()*. Do ustawienia znaku używana jest metoda *setCharAt()*.

Składnia obu metod:

char charAt(**int** *where*)

void setCharAt(**int** *where*, **char** *ch*)

```
StringBuffer sb = new StringBuffer("Witaj Świecie");
System.out.println("Bufor przed = " + sb);
System.out.println("charAt(1) przed = " + sb.charAt(1));
sb.setCharAt(1, 'y');
sb.setLength(2);
System.out.println("Bufor po = " + sb);
System.out.println("charAt(1) po = " + sb.charAt(1));
```

```
run:  
Bufor przed = Witaj Świecie  
charAt(1) przed = i  
Bufor po = Wy  
charAt(1) po = y
```

Metoda *getChars()*

Do skopiowania podciągu obiektu *StringBuffer* do tablicy służy metoda *getChars()*. Składnia:

```
void getChars( int sourcesStart, int sourceEnd,  
               char target[], int targetStart)
```

Metoda *append()*

Metoda *append()* dołącza tekst reprezentacji danej dowolnego typu do końca ciągu znaków z wywołanego obiektu *StringBuffer*. Składnia metod:

StringBuffer append(**String** *str*)

StringBuffer append(**int** *num*)

StringBuffer append(**Object** *obj*)

```
int k = 53;  
StringBuffer sb = new StringBuffer(30);  
String s =  
    sb.append("k = ").append(k).toString();  
System.out.println(s);
```

run:
k = 53

Metoda *insert()*

Metoda *insert()* wstawia jeden ciąg znaków do drugiego. Składnia:

StringBuffer insert(**int** *index*, **String** *str*)

StringBuffer insert(**int** *index*, **char** *ch*)

StringBuffer insert(**int** *index*, **Object** *obj*)

```
StringBuffer bs = new  
    StringBuffer("To jest ciąg znaków");  
bs.insert(8, "zmodyfikowany ");  
System.out.println(bs);
```

run:

To jest zmodyfikowany ciąg znaków

Metoda *reverse()*

Metoda *reverse()* odwraca kolejność znaków w obiekcie *StringBuffer*.
Składnia:

StringBuffer reverse()

```
StringBuffer bs = new  
    StringBuffer("aąbcćdeęf");  
bs.reverse();  
System.out.println(bs);
```

run:

fęedćcba

Metoda *delete()* i *deleteCharAt()*

Do usuwania znaków w obiekcie *StringBuffer* służy metoda *delete()* i *deleteCharAt()*. Składnia metod:

StringBuffer delete(**int** *startIndex*, **int** *endIndex*)

StringBuffer deleteAt(**int** *loc*)

```
StringBuffer bs = new StringBuffer("To jest test.");  
bs.delete(2, 7);  
System.out.println("Po usunięciu: " + bs);  
bs.deleteCharAt(0);  
System.out.println("Po usunięciu znaku: " + bs);
```

run:

Po usunięciu: To test.

Po usunięciu znaku: o test.

Metoda *replace()*

Metoda `replace()` zastępuje jeden podciąg znaków z obiektu `StringBuffer` innym podciągiem. Składnia:

`StringBuffer` `replace( int startIndex, int endIndex, String str)`

```
StringBuffer bs = new  
    StringBuffer("To jest test.");  
bs.replace(3, 7, "był");  
System.out.println("Po zamianie: " + bs);
```

run:

Po zamianie: To był test.

Metoda *substring()*

Do pobrania fragmentu tekstu znajdującego się w obiekcie StringBuffer służy metoda `substring()`. Składnia:

String `substring( int startIndex )`

String `substring( int startIndex, int endIndex )`

```
StringBuffer bs = new  
    StringBuffer("To jest przykład programu");  
String s = bs.substring(8, 16);  
System.out.println(s);
```

run:

przykład

Wybrane metody klasy *StringBuffer*

Metoda	Opis
StringBuffer <code>appendCodePoint(int ch)</code>	Dodaje znak Unicode do końca wywołanego obiektu.
<code>int codePoint(int index)</code>	Zwraca znak Unicode z podanego położenia.
<code>int indexOf(String str)</code>	Poszukuje w obiekcie pierwszego wystąpienia str. Zwraca jego indeks albo -1.
<code>int indexOf (String str, int startIndex)</code>	Poszukuje w obiekcie pierwszego wystąpienia str od miejsca startIndex.
<code>int lastIndexOf(String str)</code>	Poszukuje w obiekcie ostatnie wystąpienia str. Zwraca jego indeks albo -1.

Wyrażenia regularne

Pracując z łańcuchami musimy kontrolować, czy spełniają one określone kryteria zwane wzorcami. Jeżeli np. nasza aplikacja wymaga od użytkownika podania adresu e-mail, tak żeby można było wysłać wiadomość z potwierdzeniem, należy sprawdzić czy użytkownik wprowadził poprawnie adres. Przeprowadzenie kontroli za pomocą porównywania znak po znaku jest bardzo żmudne. Sprawdzenia poprawności możemy dokonać wykorzystując klasę wyrażen regularnych. Za pomocą wyrażen regularnych możemy definiować wzorce i porównywać je z zadany tekst. W środowisku Java znajdziemy klasę *Pattern* i *Matcher* pozwalającą na pracę z wyrażeniami regularnymi. Klasa *Pattern* służy do definiowania wyrażen, które są dopasowywane do sekwencji znaków za pomocą klasy *Matcher*.

Składnia wyrażeń regularnych

Ogólnie wyrażenie regularne składa się ze zwykłych znaków, ich klas (zestawów), znaków zastępczych i kwantyfikatorów. Zwykły znak jest dopasowywany bezpośrednio. Przykładowo do wzorca "ab" jedynie pasującą sekwencją jest "ab". Znaki specjalne takie jak nowej linii są opisane przez kombinacje znaków ucieczki z innymi symbolami np. "\n" – znak nowej linii.

Klasa znaków oznacza ich zbiór. Jest ona określona poprzez umieszczenie znaków w nawiasach klamrowych np. [ab] pasuje do znaków a lub b. Aby określić odwrotność zbioru, należy użyć znaku ^ np. [^ab] pasuje do wszystkich znaków za wyjątkiem a i b. Zakres znaków określamy za pomocą myślnika np. [1-9] pasuje do wszystkich cyfr, od 1 do 9.

Podstawowym znakiem zastępczym jest kropka, która pasuje do wszystkich innych znaków np. wzorzec "." pasuje do wszystkich sekwencji jednoliterowych.

Kwantyfikator określa, ile sekwencji pasujących do wzorca ma zostać znalezionych. Może on mieć jedną z następujących postaci:

- Znak + określa 1 lub więcej
- Znak * określa 0 lub więcej
- Znak ? Określa 0 lub 1

Przykładowo wzorzec "x+" pasuje m.in. do sekwencji: "x ", "xx" " i "xxx".

Wyszukiwanie tekstu

Żeby skorzystać z klasy *regex*, należy zaimportować przestrzeń nazw poleceniem:

```
import java.util.regex.*;
```

W poniższym przykładzie zostaje utworzony wzorzec, zwracający sekwencję "Java". Następnie jest tworzony do niego obiekt typu *Matcher*, zawierający taką samą sekwencję wejściową, po czym zostaje wywołana metoda *metches()*, sprawdzająca, czy sekwencja wejściowa pasuje do wzorca. Ponieważ jest ona identyczna, metoda zwraca *true*.

```
package b4b1;
import java.util.regex.*;
public class B4b1 {
    public static void main(String[] args) {
        String s = "Java";
        Pattern pat = Pattern.compile("Java");
        // Utworzenie wzorca.
        Matcher mat = pat.matcher(s);
        // Utworzenie obiektu dopasowania.
        boolean f = mat.matches();
        // Wywołanie metody sprawdzającej dopasowanie.
        if(f) System.out.println("Pasuje");
        // Sprawdzenie dopasowania.
        else System.out.println("Nie pasuje");
    }
}
```

run:
Pasuje

```
String s ="Szukam słowa Java w tekście";  
// Łancuch do przeszukania.  
Pattern pat = Pattern.compile("Java");  
// Utworzenie wzorca.  
Matcher mat = pat.matcher(s);  
// Utworzenie obiektu dopasowania.  
boolean f =mat.matches();  
// Wywołanie metody sprawdzającej dopasowanie.  
if(f) System.out.println("Pasuje");  
// Sprawdzenie dopasowania.  
else System.out.println("Nie pasuje");
```



run:

Nie pasuje

BUILD SUCCESSFUL (total time: 0 seconds)

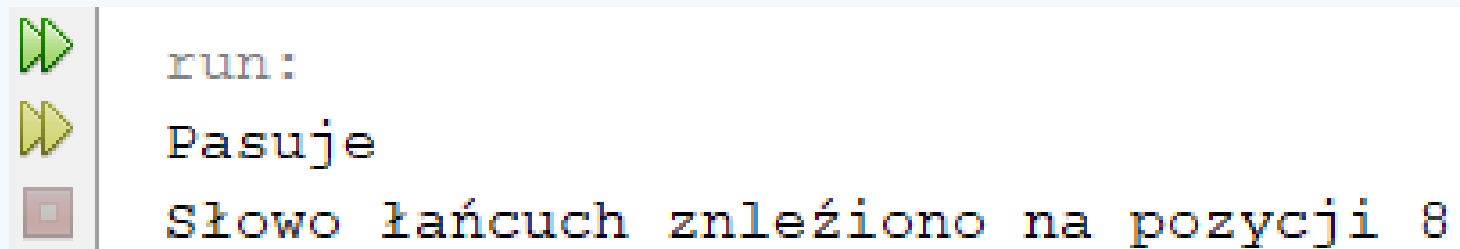
Przykłady dopasowania wzorca

Druga wersja programu zostaje utworzony nowy obiekt *Matcher* do przeszukania zawierający tekst: "Szukam słowa Java w tekście". W tym przypadku nie następuje dopasowanie do wzorca, ponieważ wzorzec jest inny niż sekwencja. Należy pamiętać, że funkcja *matches* zwraca wartość *true* tylko wówczas , gdy sekwencja wejściowa jest dokładnie tak sama jak wzorzec. Gdy pasuje do niej fragment, zwracana jest wartość *false*.

Aby sprawdzić, czy sekwencja wejściowa zawiera fragment pasujący do wzorca, należy użyć metody *find()*. Została ona wykorzystana w następnym programie.

```
String s ="To jest łańcuch znaków";
Pattern pat = Pattern.compile("łańcuch");
// Utworzenie wzorca.
Matcher mat = pat.matcher(s);
// Utworzenie obiektu dopasowania.
    if(mat.find()) System.out.println("Pasuje");
// Sprawdzenie dopasowania metodą find().
    else System.out.println("Nie pasuje");
    mat = pat.matcher(s);
// Ponowne utworzenie obiektu dopasowania
    if(mat.find())
// Przeszukanie sekwencji.
    System.out.println
        ("Słowo łańcuch znaleziono na pozycji "
            + mat.start());
```

Działanie programu



```
run:  
Pasuje  
Słowo łańcuch znaleziono na pozycji 8
```

Metoda może zostać wykorzystana również do odnajdowania w sekwencji wejściowej wielokrotnych wystąpień wzorca, ponieważ każde jej wywołanie rozpoczyna operację dopasowania w miejscu, w której skończyła się poprzednia. Następny przykład odnajduje dwa wystąpienia wzorca "łańcuch".

```
String s =  
    "To jest łańcuch znaków - długi łańcuch znaków.";  
Pattern pat = Pattern.compile("łańcuch");  
// Utworzenie wzorca.  
Matcher mat = pat.matcher(s);  
// Utworzenie obiektu dopasowania.  
    while(mat.find())  
// Przeszukanie sekwencji.  
        System.out.println  
            ("Słowo łańcuch znaleziono na pozycji "  
                + mat.start());
```



run:



Słowo łańcuch znaleziono na pozycji 8



Słowo łańcuch znaleziono na pozycji 31

Za pomocą operatora wyrażeń regularnych możliwe jest definiowanie złożonych wzorców wyszukiwania. Jeżeli w ciągu znaków powinno znaleźć się słowo Czerwony lub Żółty, możemy skorzystać z operatora |.

```
String s = "Czerwony kwiat";  
Pattern pat = Pattern.compile("Czerwony|Żółty");  
// Utworzenie wzorca.  
Matcher mat = pat.matcher(s);  
// Utworzenie obiektu dopasowania.  
while(mat.find())  
// Przeszukanie sekwencji.  
    System.out.println("Słowo " + mat.group() +  
        " znaleziono na pozycji " + mat.start());
```

run:

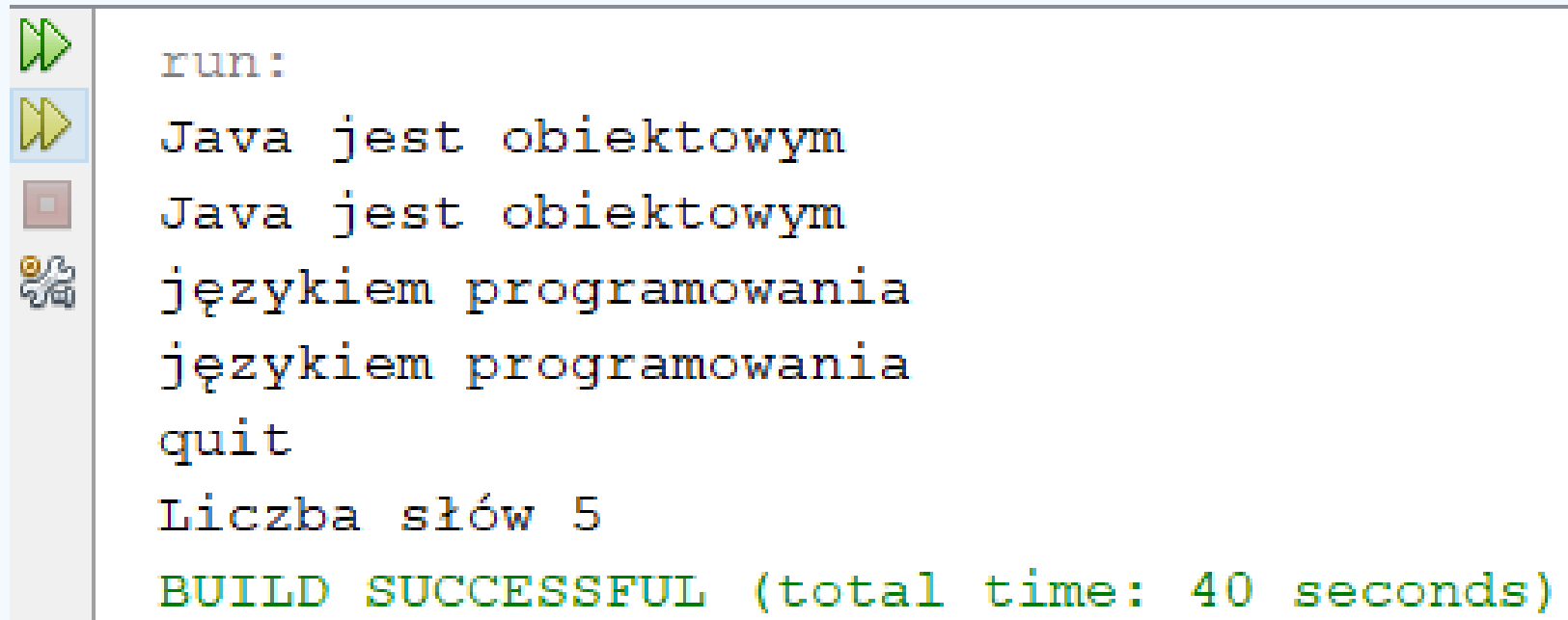
Słowo Czerwony znaleziono na pozycji 0

Zliczanie słów

```
Scanner in = new Scanner(System.in, "Cp1250");
Pattern pat = Pattern.compile("\\s\\w");
String s = "";
int t = 0;
while(!(s=in.nextLine()).equals("quit")) {
    String d = " " + s;
    System.out.println(s);
    Matcher mat = pat.matcher(d);
    while(mat.find()) t++;
}
System.out.println("Liczba słów " + t);
```

Działanie programu

Program wczytuje linie z konsoli użytkownika do momentu wpisania "quit" na początku linii . Wynik działania programu jest widoczny na rysunku poniżej.

A screenshot of an IDE console window. On the left side, there is a vertical toolbar with four icons: a green play button, a yellow play button, a red square stop button, and a grey bug icon. The main area of the console displays the following text:

```
run:  
Java jest obiekowym  
Java jest obiekowym  
językiem programowania  
językiem programowania  
quit  
Liczba słów 5  
BUILD SUCCESSFUL (total time: 40 seconds)
```

W tabeli zostały umieszczone operatory używane do konstrukcji wzorców.

Operator	Opis
.	Dopasowuje jeden dowolny znak.
[]	Dopasowuje dowolny znak spośród znaków podanych w nawiasach klamrowych.
[^]	Dopasowuje dowolny znak spośród znaków niewpisanych w nawiasach klamrowych.
?	Dopasowuje jeden dowolny znak, o ile taki istnieje.
*	Wielokrotnie dopasowuje zadeklarowany element, o ile taki istnieje.
+	Jeden lub więcej razy dopasowuje zadeklarowany element.
{n}	Dokładnie n razy dopasowuje zadeklarowany element.
{n,}	Przynajmniej n razy dopasowuje zadeklarowany element.

Operator	Opis
{n,N}	Przynajmniej n razy, ale nie więcej niż N razy dopasowuje zadeklarowany element.
^	Dopasowuje na początku wiersza.
\$	Dopasowuje na końcu wiersza.
\<	Dopasowuje na początku słowa.
\>	Dopasowuje na końcu słowa.
\b	Dopasowuje na początku lub na końcu słowa.
\B	Dopasowuje w środku słowa.
\d	Skrót opisujący cyfry (0-9).
\w	Skrót opisujący znaki alfanumeryczne (litery i cyfry).
\s	Skrót opisujące znaki białe.

Wzorzec sprawdzający poprawność zapisu daty

Innym typowym wzorcem wyszukiwania jest wzorzec sprawdzający ciąg znaków zawierających datę w formacie dd-mm-yyyy , to wzorzec wyszukiwania można zdefiniować następująco:

```
"([1-9]|0[1-9]|[12][0-9]|3[01])[-//.](0[1-9]|1[012])[-//.](19|20)\\d\\d"
```

W tak dopasowanym wzorcu można dopasować datę od 01-01-1900 do 31-12-2099. Dopuszczone jest podanie daty w postaci : 2-01-2014. Wzorzec ten dopuszcza również użycie znaku . albo / zamiast znaku – w zapisie daty.

```
String s = "2-01-2014";
Pattern pat = Pattern.compile
    ("([1-9]|0[1-9]|[12][0-9]|3[01])[-//.]" +
     "(0[1-9]|1[012])[-//.](19|20)\\d\\d");
// Utworzenie wzorca.
Matcher mat = pat.matcher(s);
// Utworzenie obiektu dopasowania.
if(mat.find()) System.out.println("To jest data");
// Przeszukanie sekwencji.
else System.out.println("To nie jest data");
```



run:



To jest data



BUILD SUCCESSFUL (total time: 0 seconds)